

Design And Analysis of Algorithms

Muhammad Danish – 22i-1305

Shahmeer Ahmad – 22i-1048

Question#1

Pseudocode:

Function Structure(N, prev, dp):

 sum = 0

 Ntemp = N

 If prev is not equal to -1 and dp[N][prev] is not equal to -1:

 Return dp[N][prev]

 If N is less than prev and prev is not equal to -1:

 sum = 1

 x = 1

 N = N - 1

 While N is greater than 1:

 If prev is equal to -1 or N is less than prev:

 sum = sum + Structure(x, N, dp)

 N = N - 1

 x = x + 1

 If prev is not equal to -1:

dp[Ntemp][prev] = sum

Return sum

Function main():

Read N from input

Initialize dp array of size N x N with all elements set to -1

Print Structure(N, -1, dp)

Analysis:

Function Structure:

- The time complexity of this function depends on the value of N.
- The function iterates over N in a loop, making recursive calls to itself with decreasing values of N.
- At each recursive call, the function performs a constant number of operations.
- Therefore, the time complexity of this function can be expressed as $O(N^2)$ in the worst case, considering the nested loop structure.

Main Function:

- The main function only initializes the dp array and calls the Structure function.
- Initialization of the dp array takes $O(N^2)$ time.
- The time complexity of the Structure function dominates the overall time complexity.

Thus, the overall time complexity of the program is $O(N^2)$.

Question#2

Pseudocode:

Function findOptimalCuts(N, M, depots):

Initialize cuts array of size M with all elements set to -1

Initialize cutTotal array of size M+1 with all elements set to -1

Function computeProductSum(start, end):

 sum = 0

 total = 0

 For i from end to start:

 total = total + depots[i] * sum

 sum = sum + depots[i]

 Return total

Function readjustTotal(cutIndex):

 start = 0

 end = 0

 If cutIndex is 0:

 start = 0

 Else:

 start = cuts[cutIndex - 1] + 1

 end = cuts[cutIndex]

 cutTotal[cutIndex] = computeProductSum(start, end)

 If cutIndex is M - 1 or cuts[cutIndex + 1] is -1:

 end = N - 1

 Else:

 end = cuts[cutIndex + 1]

 start = cuts[cutIndex] + 1

 cutTotal[cutIndex + 1] = computeProductSum(start, end)

Function insertionCut(num):

i = M - 1

While cuts[i-1] is -1:

i = i - 1

While i > 0 and cuts[i-1] > num:

cuts[i] = cuts[i-1]

cutTotal[i + 1] = cutTotal[i]

i = i - 1

cuts[i] = num

Return i

Function minStrategicValue(start, end, M):

sum = computeProductSum(start, end)

If M is 0:

Return

minVal = sum

If start + 1 is equal to end:

cut = start

Else:

For cut from start to end - 1:

currentCutSum = computeProductSum(start, cut) + computeProductSum(cut + 1, end)

If currentCutSum < minVal:

minVal = currentCutSum

Else:

Break

cut = cut - 1

index = insertionCut(cut)

readjustTotal(index)

M = M - 1

If M is 0:

Return

max = 0

maxIndex = -1

For i from 1 to N:

 If cutTotal[i] > max:

 max = cutTotal[i]

 maxIndex = i

start = maxIndex - 1

If maxIndex is N:

 end = N - 1

Else:

 end = cuts[maxIndex]

If start is greater than or equal to 0:

 start = cuts[start]

minStrategicValue(start + 1, end, M)

Call minStrategicValue(1, N, M)

total = Sum of elements in cutTotal

Print total

Function main():

 While True:

 Read N, M from input

 If N is 0 and M is 0:

 Break

Read depots from input

Call findOptimalCuts(N, M, depots)

Analysis:

1. Space of Sub-problems:

The space of sub-problems is defined by the parameterization of the minStrategicValue function, which takes three parameters:

- start: the starting index of the current segment
- end: the ending index of the current segment
- M: the number of cuts remaining

The overall goal is to minimize the Strategic Value of the remaining railroad, which is reflected in the sub-problems by trying different cut positions and updating the minimum value.

2. Recursive Formula:

The recursive formula to compute the values of the sub-problems is:

$$S(i, j, k) = \min_{i \leq c < j} (S(i, c, k-1) + S(c, j, 0))$$

3. Bottom-up Implementation

To solve the sub-problems in a bottom-up manner, we would iterate over the possible cut positions in increasing order. For each cut position, we would compute the minimum Strategic Value for the left and right segments, and update the cut_total array accordingly.

4. Time complexity:

- computeProductSum: $O(N)$
- readjust_total: $O(N)$ (linear complexity)
- insertion_cut: $O(N)$ (insertion sort)
- minStrategicValue: $O(M \cdot N^2)$ (recursive function with memoization)
- main: $O(N)$ (reading input and printing output)

The overall time complexity is $O(M \cdot N^2)$ due to the recursive function minStrategicValue.

5. Space complexity:

- depots: $O(N)$
- cuts: $O(M)$
- cut_total: $O(M+1)$

The overall space complexity is $O(N+M)$.

6. Memoization:

- The cut_total array is used to store the product sum of depots between cuts, which is a form of memoization.
- The minStrategicValue function uses this memoization to avoid redundant computations.

7. Recursion:

- The minStrategicValue function is recursive, with a depth of up to M (the number of cuts).
- The function has a base case when M is 0, and it returns immediately.

8. Dynamic programming:

- The code uses dynamic programming to store the product sum of depots between cuts in the cut_total array.
- This allows for efficient computation of the Strategic Value for each segment.

9. Optimization:

- The code uses insertion sort for inserting new cuts, which has a time complexity of $O(N)$.
- The readjust_total function has a linear complexity, which is efficient for updating the cut_total array.

Overall, the code has a time complexity of $O(M \cdot N^2)$ and a space complexity of $O(N+M)$.

Question#3

Pseudocode

Function findSegments(arr, minDP, N):

max = -1

cmnp = 0

next = 1

For i from 0 to N - 1:

val = arr[i]

If val > max:

max = val

If next == val:

Increment next by 1

If val == cmnp:

cmnp = next

Increment next by 1

If next == max:

Increment next by 1

Store cmnp in minDP[0][i]

max = -1

cmnp = 0

next = 1

For i from N - 1 to 0:

val = arr[i]

If val > max:

max = val

If next == val:

Increment next by 1

If val == cmnp:

cmnp = next

Increment next by 1

If next == max:

Increment next by 1

Store cmnp in minDP[1][i]

For i from 0 to N - 2:

If minDP[0][i] == minDP[1][i + 1]:

Return i + 1

Return -1

Function main():

Read N from input

Initialize arr as an array of size N

Initialize minDP as a 2D array of size 2 x N

For i from 0 to N - 1:

Read arr[i] from input

Set minDP[0][i] to -1

Set minDP[1][i] to -1

partition = findSegments(arr, minDP, N)

If partition != -1:

Print 2

Print "1 partition"

Print "partition + 1 N"

Else:

Print partition

Analysis:

The time complexity of your code is $O(N)$, where N is the number of elements in the array.

Here's a breakdown of the time complexity analysis:

- The first loop in the findSegments function iterates N times, and each iteration performs a constant amount of work. Therefore, the time complexity of this loop is $O(N)$.
- The second loop in the findSegments function also iterates N times, and each iteration performs a constant amount of work. Therefore, the time complexity of this loop is also $O(N)$.
- The final loop in the findSegments function iterates at most $N-1$ times, and each iteration performs a constant amount of work. Therefore, the time complexity of this loop is also $O(N)$.
- The total time complexity of the findSegments function is the sum of the time complexities of the three loops, which is $O(N) + O(N) + O(N) = O(3N) = O(N)$.
- The main function performs a constant amount of work, so its time complexity is $O(1)$.

Therefore, the overall time complexity of your code is $O(N)$.

Question#4

Pseudocode:

Function MaxPower(M, N, arr):

$max1 = Data(-1, 0)$

$max2 = Data(-1, 0)$

$pmax1 = Data(0, -1)$

$pmax2 = Data(0, -1)$

For j from $N - 1$ to 0 :

$max1.value = -1$

$max2.value = -1$

 For i from 0 to $M - 1$:

num = arr[i][j]

If i is not equal to pmax1.pos:

Increment num by pmax1.value

Else:

Increment num by pmax2.value

If num is greater than or equal to max1.value:

max2.value = max1.value

max2.pos = max1.pos

max1.value = num

max1.pos = i

Else If num is greater than or equal to max2.value:

max2.value = num

max2.pos = i

If max1.value is greater than pmax1.value:

If pmax1.value is greater than max2.value:

pmax2.value = pmax1.value

pmax2.pos = pmax1.pos

Else:

pmax2.value = max2.value

pmax2.pos = max2.pos

pmax1.value = max1.value

pmax1.pos = max1.pos

Else If max1.value is greater than pmax2.value:

pmax2.value = max1.value

pmax2.pos = max1.pos

Return max1.value

Function main():

Read M and N from input

Initialize arr as a 2D array of size M x N

For i from 0 to M - 1:

Initialize arr[i] as an array of size N

For i from 0 to M - 1:

For j from 0 to N - 1:

Read arr[i][j] from input

max = 0

If M is equal to 1:

For i from 0 to N - 1:

val = arr[0][i]

If val is greater than max:

Set max to val

Else:

Set max to MaxPower(M, N, arr)

Print max

Analysis:

The time complexity of this code is $O(M*N)$, where M is the number of rows and N is the number of columns in the input array.

- The outer loop in the MaxPower function iterates N times (from $N-1$ to 0).
- The inner loop iterates M times (from 0 to $M-1$).
- The total number of iterations is therefore $N*M$.
- The time complexity of the MaxPower function is $O(N*M)$.
- The main function has a constant time complexity of $O(1)$ because it only performs a few operations that do not depend on the input size.

The overall time complexity of the code is therefore $O(N*M)$.